

Autentikasi dan Keamanan Web API: Registrasi, Login, JWT, Role-Based Access, dan Token Blacklisting di Go

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Keamanan adalah pilar utama dalam pengembangan REST API. Di era modern, sistem autentikasi berbasis token seperti **JWT (JSON Web Token)** menjadi standar industri untuk menjaga pertukaran data tetap aman dan tanpa status (*stateless*). Selain mengidentifikasi pengguna, sistem harus membedakan hak akses pengguna menggunakan konsep **RBAC (Role-Based Access Control)** serta mengizinkan pengguna untuk keluar dari sistem secara aman melalui mekanisme **Token Blacklisting**.

Modul ini akan membahas keamanan penyimpanan sandi menggunakan *Bcrypt*, struktur dan klaim token JWT, pembuatan middleware autentikasi, serta implementasi alur registrasi, login, logout berbasis blacklist, dan endpoint profil pengguna (*Auth Me*).

Sesi 1: Registrasi, Login & Hashing Kata Sandi dengan Bcrypt (45 Menit)

Menyimpan kata sandi dalam bentuk teks murni (*plain text*) di database adalah pelanggaran keamanan yang sangat fatal. Jika database bocor, seluruh akun pengguna akan langsung terekspos.

1. Mengenal Bcrypt Hashing

Bcrypt adalah algoritma hashing satu arah yang dirancang khusus untuk mengamankan kata sandi. Bcrypt menambahkan nilai acak (*Salt*) secara otomatis dan memiliki faktor biaya (*Work Factor/Cost*) yang membuat proses hashing sengaja melambat untuk mencegah serangan *brute-force*.

Untuk menggunakan Bcrypt di Go, instal paket kriptografi berikut:

```
go get -u golang.org/x/crypto/bcrypt
```

- **Fungsi Hashing Kata Sandi:**

```
func HashPassword(password string) (string, error) {  
    // Cost 12 adalah standar industri yang aman dan efisien
```

```
bytes, err := bcrypt.GenerateFromPassword([]byte(password), 12)
return string(bytes), err
}
```

- **Fungsi Verifikasi Kata Sandi:**

```
func CheckPasswordHash(password, hash string) bool {
    err := bcrypt.CompareHashAndPassword([]byte(hash), []byte(password))
    return err == nil // Mengembalikan true jika cocok
}
```

2. Alur Registrasi & Login

- **Registrasi:** Menerima input data DTO (Email, Password, Role) -> Validasi -> Hash Password -> Simpan User ke DB.
- **Login:** Menerima input data DTO (Email, Password) -> Ambil data User berdasarkan Email -> Bandingkan input password dengan hash di DB -> Jika cocok, buat dan kirimkan Token JWT.

Sesi 2: JWT (JSON Web Token) & Role-Based Payload (45 Menit)

JWT (JSON Web Token) adalah standar terbuka (RFC 7519) untuk mentransmisikan informasi secara aman sebagai objek JSON. JWT terdiri dari tiga bagian yang dipisahkan oleh tanda titik (.):

1. **Header:** Algoritma hashing yang digunakan (misal: HS256) dan tipe token.
2. **Payload:** Data klaim pengguna (seperti `user_id` , `email` , dan `role`) serta waktu kedaluwarsa (`exp`).
3. **Signature:** Kunci verifikasi bertanda tangan digital untuk mendeteksi manipulasi token.

Instal pustaka JWT untuk Go:

```
go get -u github.com/golang-jwt/jwt/v5
```

Pembuatan Token JWT (Generate Token)

```
package main

import (
    "time"

    "github.com/golang-jwt/jwt/v5"
)
```

```

var JWTSecretKey = []byte("KunciRahasiaNegaraSangatRahasia999!")

type JWTClaims struct {
    UserID uint   `json:"user_id"`
    Email  string  `json:"email"`
    Role   string  `json:"role"` // "ADMIN" atau "USER"
    jwt.RegisteredClaims
}

func GenerateJWT(userID uint, email string, role string) (string, error) {
    claims := JWTClaims{
        UserID: userID,
        Email:  email,
        Role:   role,
        RegisteredClaims: jwt.RegisteredClaims{
            // Token kedaluwarsa dalam 1 jam
            ExpiresAt: jwt.NewNumericDate(time.Now().Add(1 * time.Hour)),
            IssuedAt:   jwt.NewNumericDate(time.Now()),
        },
    }

    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString(JWTSecretKey)
}

```

Sesi 3: Authorization Middleware & Token Blacklist Logout (45 Menit)

1. Middleware Autentikasi JWT

Middleware bertugas mengekstrak token dari header `Authorization: Bearer <token>`, memvalidasi masa berlaku dan tanda tangan digitalnya, lalu menyimpan data klaim pengguna ke dalam `gin.Context` agar dapat diakses oleh handler utama.

```

func AuthMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        authHeader := c.GetHeader("Authorization")
        if authHeader == "" {
            c.AbortWithStatusJSON(401, gin.H{"error": "Header otorisasi kosong"})
            return
        }

        // Memisahkan teks "Bearer <token>"
        tokenParts := strings.Split(authHeader, " ")
    }
}

```

```

        if len(tokenParts) != 2 || tokenParts[0] != "Bearer" {
            c.AbortWithStatusJSON(401, gin.H{"error": "Format otorisasi
harus Bearer <token>"})
            return
        }

        tokenString := tokenParts[1]

        // 1. Cek apakah token masuk dalam daftar Blacklist (Logout)
        if isTokenBlacklisted(tokenString) {
            c.AbortWithStatusJSON(401, gin.H{"error": "Token sudah tidak
berlaku (telah logout)"})
            return
        }

        // 2. Parse dan validasi token JWT
        claims := &JWTClaims{}
        token, err := jwt.ParseWithClaims(tokenString, claims, func(t
*jwt.Token) (interface{}, error) {
            return JWTSecretKey, nil
        })

        if err != nil || !token.Valid {
            c.AbortWithStatusJSON(401, gin.H{"error": "Token tidak valid
atau kedaluwarsa"})
            return
        }

        // 3. Simpan data klaim ke context Gin
        c.Set("userID", claims.UserID)
        c.Set("email", claims.Email)
        c.Set("role", claims.Role)
        c.Set("tokenString", tokenString) // disimpan untuk kebutuhan logout

        c.Next()
    }
}

```

2. Konsep Token Blacklisting (Logout)

Karena JWT bersifat *stateless* (tidak disimpan di server), server tidak bisa menghapus token secara sepihak saat pengguna memanggil endpoint Logout. Sebelum masa berlaku token habis, token tersebut secara teoritis masih valid.

- **Solusi:** Menyimpan tanda tangan (*signature*) atau string token yang telah di-logout ke dalam database blacklist bertempo (atau *Redis*) hingga waktu kedaluwarsanya habis. Setiap request yang masuk akan diperiksa silang dengan daftar blacklist ini.

Sesi 4: Praktikum Mandiri: REST API Autentikasi dengan Registrasi, Login, Logout, dan Endpoint Me (45 Menit)

Kita akan membuat API autentikasi utuh di database PostgreSQL (`ecommercedb` port 5434). Praktikum ini mencakup tabel `users` dan `blacklisted_tokens` untuk mengelola sesi secara aman.

Implementasi Kode (`main.go`)

```
package main

import (
    "errors"
    "net/http"
    "strings"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/golang-jwt/jwt/v5"
    "golang.org/x/crypto/bcrypt"
    "gorm.io/driver/postgres"
    "gorm.io/gorm"
)

// =====
// 1. DATA MODELS & DATABASE CONFIGURATION
// =====

type User struct {
    ID          uint       `gorm:"primaryKey" json:"id"`
    Email       string     `gorm:"type:varchar(100);uniqueIndex;not null" json:"email"`
    Password    string     `gorm:"type:varchar(255);not null" json:"-` // password
    Role        string     `gorm:"type:varchar(20);default:'USER'" json:"role" //
    CreatedAt   time.Time `json:"created_at"`
}

type BlacklistedToken struct {
    ID          uint       `gorm:"primaryKey"`
    TokenString string     `gorm:"type:text;uniqueIndex;not null"`
    ExpiredAt   time.Time `gorm:"not null"`
}

var db *gorm.DB
var JWTSecret = []byte("KunciSuperRahasiaDanPanjangUntukKeamanan999!")

func initDatabase() {
    dsn := "host=localhost user=postgres password=secret45 dbname=ecommercedb"
```

```

port=5434 sslmode=disable TimeZone=Asia/Jakarta"
    var err error
    db, err = gorm.Open(postgres.Open(dsn), &gorm.Config{})
    if err != nil {
        panic("Gagal menghubungkan database: " + err.Error())
    }
    _ = db.AutoMigrate(&User{}, &BlacklistedToken{})
}

// =====
// 2. JWT UTILITIES
// =====

type JWTClaims struct {
    UserID uint   `json:"user_id"`
    Email  string  `json:"email"`
    Role   string  `json:"role"`
    jwt.RegisteredClaims
}

func GenerateToken(user User) (string, error) {
    claims := JWTClaims{
        UserID: user.ID,
        Email:  user.Email,
        Role:   user.Role,
        RegisteredClaims: jwt.RegisteredClaims{
            ExpiresAt: jwt.NewNumericDate(time.Now().Add(1 * time.Hour)),
            IssuedAt:  jwt.NewNumericDate(time.Now()),
        },
    }
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString(JWTSecret)
}

func isTokenBlacklisted(tokenStr string) bool {
    var count int64
    db.Model(&BlacklistedToken{}).Where("token_string = ?",
tokenStr).Count(&count)
    return count > 0
}

// =====
// 3. MIDDLEWARES & AUTHENTICATION DTOs
// =====

type RegisterInput struct {
    Email    string `json:"email" binding:"required,email"`
    Password string `json:"password" binding:"required,min=6"`
    Role     string `json:"role" binding:"omitempty,oneof=USER ADMIN"`
}

type LoginInput struct {
    Email string `json:"email" binding:"required,email"`

```

```

        Password string `json:"password" binding:"required"`
    }

    func AuthMiddleware() gin.HandlerFunc {
        return func(c *gin.Context) {
            authHeader := c.GetHeader("Authorization")
            if authHeader == "" {
                c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
"Header Authorization kosong"})
                return
            }

            parts := strings.Split(authHeader, " ")
            if len(parts) != 2 || parts[0] != "Bearer" {
                c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
"Format token wajib Bearer <token>"})
                return
            }

            tokenStr := parts[1]

            // Periksa Blacklist
            if isTokenBlacklisted(tokenStr) {
                c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
"Sesi telah berakhir (Anda telah logout)"})
                return
            }

            // Validasi JWT
            claims := &JWTClaims{}
            token, err := jwt.ParseWithClaims(tokenStr, claims, func(t *jwt.Token)
(interface{}, error) {
                return JWTSecret, nil
            })

            if err != nil || !token.Valid {
                c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
"Token tidak valid atau telah kedaluwarsa"})
                return
            }

            // Simpan claims ke context
            c.Set("userID", claims.UserID)
            c.Set("email", claims.Email)
            c.Set("role", claims.Role)
            c.Set("tokenStr", tokenStr)
            c.Set("tokenExp", claims.ExpiresAt.Time)

            c.Next()
        }
    }

    // =====

```

```
// 4. CONTROLLERS & ENDPOINTS
// =====

func main() {
    initDatabase()

    r := gin.Default()

    authGroup := r.Group("/api/v1/auth")
    {
        // POST /api/v1/auth/register - Registrasi Akun
        authGroup.POST("/register", func(c *gin.Context) {
            var input RegisterInput
            if err := c.ShouldBindJSON(&input); err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error":
err.Error()})

                return
            }

            // Cek duplikasi email
            var existing User
            errEmail := db.Where("email = ?",
input.Email).First(&existing).Error
            if errEmail == nil {
                c.JSON(http.StatusConflict, gin.H{"error": "Email
sudah terdaftar di sistem"})

                return
            }

            // Hash Password
            hashedPassword, errHash :=
bcrypt.GenerateFromPassword([]byte(input.Password), 12)
            if errHash != nil {
                c.JSON(http.StatusInternalServerError, gin.H{"error":
"Gagal memproses kata sandi"})

                return
            }

            role := input.Role
            if role == "" {
                role = "USER" // default role
            }

            newUser := User{
                Email:    input.Email,
                Password: string(hashedPassword),
                Role:     role,
            }

            if err := db.Create(&newUser).Error; err != nil {
                c.JSON(http.StatusInternalServerError, gin.H{"error":
err.Error()})

                return
            }
        })
    }
}
```

```

    }

    c.JSON(http.StatusCreated, gin.H{
        "success": true,
        "message": "Registrasi pengguna berhasil!",
        "data":    newUser,
    })
})

// POST /api/v1/auth/login - Login Pengguna
authGroup.POST("/login", func(c *gin.Context) {
    var input LoginInput
    if err := c.ShouldBindJSON(&input); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error":
err.Error()})

        return
    }

    var user User
    if err := db.Where("email = ?",
input.Email).First(&user).Error; err != nil {
        c.JSON(http.StatusUnauthorized, gin.H{"error": "Email
atau kata sandi salah"})

        return
    }

    // Bandingkan Password Hash
    errPassword :=
bcrypt.CompareHashAndPassword([]byte(user.Password), []byte(input.Password))
    if errPassword != nil {
        c.JSON(http.StatusUnauthorized, gin.H{"error": "Email
atau kata sandi salah"})

        return
    }

    // Generate JWT Token
    token, errToken := GenerateToken(user)
    if errToken != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error":
"Gagal menerbitkan token"})

        return
    }

    c.JSON(http.StatusOK, gin.H{
        "success": true,
        "message": "Login berhasil!",
        "token":   token,
    })
})

// Endpoint terproteksi login
authGroup.Use(AuthMiddleware())
{

```

```

// GET /api/v1/auth/me - Profil Pengguna Aktif
authGroup.GET("/me", func(c *gin.Context) {
    userID := c.MustGet("userID").(uint)

    var user User
    if err := db.First(&user, userID).Error; err != nil {
        c.JSON(http.StatusNotFound, gin.H{"error":
"Pengguna tidak ditemukan"})

        return
    }

    c.JSON(http.StatusOK, gin.H{
        "success": true,
        "data":    user,
    })
})

// POST /api/v1/auth/logout - Logout & Blacklist Token
authGroup.POST("/logout", func(c *gin.Context) {
    tokenStr := c.MustGet("tokenStr").(string)
    tokenExp := c.MustGet("tokenExp").(time.Time)

    // Masukkan token ke tabel Blacklist
    blacklist := BlacklistedToken{
        TokenString: tokenStr,
        ExpiredAt:   tokenExp,
    }

    if err := db.Create(&blacklist).Error; err != nil {
        c.JSON(http.StatusInternalServerError,
gin.H{"error": "Gagal memproses logout"})

        return
    }

    c.JSON(http.StatusOK, gin.H{
        "success": true,
        "message": "Logout berhasil. Token
dinonaktifkan secara permanen.",
    })
})

}

// Rute Proteksi Admin Khusus
adminGroup := r.Group("/api/v1/admin")
adminGroup.Use(AuthMiddleware())
{
    // GET /api/v1/admin/dashboard
    adminGroup.GET("/dashboard", func(c *gin.Context) {
        role := c.MustGet("role").(string)

        // Pengecekan otorisasi manual tingkat rute (RBAC)
        if role != "ADMIN" {

```

```

        c.AbortWithStatusJSON(http.StatusForbidden, gin.H{
            "error": "Akses ditolak: Resource ini hanya
diperuntukkan bagi Administrator",
        })
        return
    }

    c.JSON(http.StatusOK, gin.H{
        "success": true,
        "message": "Selamat datang di Panel Dashboard Admin!",
    })
})

r.Run(":8080")
}

```

Sesi 5: Soal Latihan Praktik E-Commerce (Mandiri)

Selesaikan dua latihan pemrograman middleware otorisasi di bawah ini untuk memperkuat pemahaman Anda mengenai arsitektur keamanan RBAC di Go.

Latihan 1: Pembuatan Middleware Otorisasi Hak Akses (`RequireRole`)

Deskripsi Soal: Pengecekan role secara manual di dalam handler seperti pada Sesi 4 (`if role != "ADMIN"`) kurang efisien jika diterapkan pada banyak rute. Buatlah middleware penentu hak akses yang bersih.

Kriteria Implementasi:

1. Buatlah sebuah fungsi middleware bernama `RequireRole(allowedRoles ...string) gin.HandlerFunc`.
2. Di dalam middleware tersebut:
 - Ekstrak parameter `"role"` dari context Gin yang sebelumnya telah disimpan oleh `AuthMiddleware`.
 - Periksa apakah role pengguna terdaftar di dalam daftar `allowedRoles` yang diperbolehkan.
 - Jika tidak terdaftar, hentikan request dengan `c.AbortWithStatusJSON` status 403 Forbidden.
3. Terapkan middleware baru ini untuk memproteksi endpoint admin secara lebih elegan:

```
adminGroup.GET("/dashboard", RequireRole("ADMIN"), handleDashboard)
```

Latihan 2: Pembersihan Token Blacklist Berkala (Database Garbage Collection)

Deskripsi Soal: Tabel `blacklisted_tokens` akan terus membengkak seiring bertambahnya aktivitas logout pengguna. Padahal, token yang masa kedaluwarsanya (`expired_at`) sudah lewat secara alami tidak bisa digunakan lagi oleh sistem meskipun tidak di-blacklist.

Kriteria Implementasi:

1. Buatlah fungsi latar belakang (*background worker*) menggunakan Go Routine dan Timer / Ticker(`time.NewTicker`) di dalam `main.go` .
2. Jalankan fungsi pembersihan berkala tersebut setiap **10 menit** sekali.
3. Di dalam fungsi pembersih, jalankan kueri SQL berikut untuk menghapus baris token yang telah melewati masa berlakunya:

```
DELETE FROM blacklisted_tokens WHERE expired_at < NOW();
```

4. Cetak log informasi ke console ketika database berhasil merapikan sisa sampah token blacklist tersebut.